

GAMA PLATFORM

Road Traffic Simulation

Technical Reference & Blueprint

For GAMA 1.9.x • GAML Syntax • GIS/OSM-Driven Agent-Based Modeling

Prepared as a Senior Research Engineer reference document

1. GAML Core Architecture & Syntax Rules

1.1 The Three-Tier Execution Model

Every GAMA model is built from three conceptual blocks, always declared in this order:

```
model RoadTrafficModel // (1) model declaration – must match filename convention

global { } // (2) the "world" species – setup, global vars, global reflexes

species road { } // (3..n) species declarations – agent templates
species vehicle { }
species intersection { }

experiment MainExperiment type: gui { } // (last) simulation entry point(s)
```

Execution lifecycle per cycle (tick)

- global reflexes/scheduled actions run first (unless schedules: is overridden).
- Each species' agents execute their reflex blocks in declaration order, filtered by any when: clause, for agents whose schedules: facet includes them (default = all instances, every cycle).
- aspect blocks are NOT behavior — they only run during the display refresh to draw agents. They never mutate state.
- actions are never invoked automatically — they are called explicitly (from a reflex, from init, or from another agent via ask).

1.2 Anatomy of a Species

```
species vehicle skills: [moving] {
  // ---- ATTRIBUTES (state) ----
  float speed_max <- (30 + rnd(30)) #km / #h;
  rgb color <- rnd_color(255);
  point target;

  // ---- INIT (constructor-like block, runs once per agent) ----
  init {
    speed <- speed_max;
    location <- one_of(intersection collect each.location);
  }

  // ---- REFLEX (auto-run each cycle if when: is true) ----
  reflex move when: target != nil {
    do goto target: target on: road_network;
  }

  // ---- ACTION (callable procedure, may take args, may return a value) ----
  action recompute_route(point new_target) {
    target <- new_target;
    return path_between(road_network, location, new_target);
  }

  // ---- ASPECT (pure rendering, no side effects) ----
  aspect base {
```

```

    draw triangle(5) color: color rotate: heading;
  }
}

```

1.3 Experiment Types

```

experiment RunGUI type: gui {
  parameter "Number of vehicles" var: nb_vehicles_init min: 1 max: 500 category: "Traffic";
  output { /* displays, monitors, charts – see Section 5 */ }
}

experiment RunBatch type: batch repeat: 10 keep_seed: false until: (time > 3600) {
  parameter "Number of vehicles" var: nb_vehicles_init among: [50, 100, 200];
  method exploration; // or: genetic, tabu, hill_climbing,
  simulated_annealing...
  reflex end_of_runs {
    ask simulations { write "Avg speed: " + mean_speed; }
  }
}

```

- gui: interactive, one simulation at a time, full display/UI support.
- batch: headless, runs many replications/parameter combinations for calibration or sensitivity analysis — no output/display block executes graphics, only permanent/final reflexes and file exports.

1.4 Strict GAML Dos and Don'ts

☑ DO	✗ DON'T	Why
Use <- for assignment and initialization	Use = (that's comparison in GAML)	= inside a normal statement is a boolean test, not an assignment – a silent logic bug, not a syntax error.
Use <- even inside let and attribute declarations: let x <- 5;	Write let x = 5;	Same reason.
Terminate every statement with ;	Omit semicolons (NetLogo/Python habit)	GAML is block/statement-based like Java/Pascal.
Declare typed variables: float speed <- 2.0;	Leave type off (var speed = 2.0, JS style)	GAML is statically typed; unknown exists but should be avoided for performance and correctness.
Use ask target_agent { ... } to have one agent instruct another	Directly call otherAgent.reflexName() (Java style)	GAML uses agent-communication semantics, not method calls.
Use do action_name(args); to call an action	Call action_name(args); bare, or self.action() (Java leak)	do is mandatory for invoking actions/primitives as statements.
Use species_name[index] or first(species_name) to grab agents	Use array-style .get(i) (Java leak)	Agents are accessed via GAML containers (lists, and species act as implicit lists).

☑ DO	✗ DON'T	Why
Use create species_name number: n { ... }	Use new species_name() (Java/OOP leak)	create is the GAML agent-instantiation statement.
End if/else blocks with braces; use else if	Use elif (Python leak)	GAML syntax mirrors C-family conditionals.
Use and / or / not (word operators)	Rely only on && / without knowing both forms exist	GAML supports both symbol and word boolean operators; word form is idiomatic GAML style.
Cast explicitly: int(my_float), string(my_int)	Assume implicit coercion always works	Some coercions (e.g. unknown → typed) silently fail or throw; cast defensively.
Use # unit pseudo-constants: 5 #km, 2 #s, 10 #km/#h	Hardcode raw numbers for physical quantities	GAML's unit system auto-converts to internal base units; mixing raw floats causes silent scale bugs.
Use geometry, point, list, agent, species as first-class types	Try to define classes/interfaces (OOP leak)	GAML has no class; species themselves are the type system.

1.5 Common Spatial Operators (memorize these)

```

envelope(a_shapefile)           // bounding geometry of a GIS layer, used to size the world
as_edge_graph(a_list_of_geoms) // topological graph from a list of line geometries
as_driving_graph(edges, nodes) // pre-built graph tuned for advanced_driving skill
path_between(graph, p1, p2)     // shortest path (returns a path object)
closest_to(agent_or_point, candidate_list)
agents_overlapping(self)
agents_at_distance(self, 10#m)
self distance_to other
self towards other              // heading in degrees toward another point/agent
self direction_to other
line([p1, p2])
circle(radius)
buffer(a_geometry, distance)
inter(geom1, geom2)             // intersection
first(list) / last(list)

```

2. GIS & Environment Setup

2.1 Loading GIS Files & Binding the World Envelope

The world/environment size must be derived from the GIS data, never hardcoded, so the model scales to any dataset.

```
global {
  // Declare each shapefile as a file variable
  file shape_file_roads    <- file("../includes/roads.shp");
  file shape_file_buildings <- file("../includes/buildings.shp");
  file shape_file_nodes    <- file("../includes/nodes.shp");

  // The shape of world MUST be set from envelope() of one authoritative layer
  // (commonly the roads layer, since it usually has the largest extent)
  geometry shape <- envelope(shape_file_roads);

  graph road_network;

  init {
    // Defensive checks – fail fast and loud rather than silently building an empty model
    if (shape_file_roads = nil) {
      error "FATAL: roads.shp could not be loaded – check ../includes/ path.";
    }

    create road from: shape_file_roads with: [
      lanes:: int(get("lanes")),          // maps GIS attribute "lanes" -> agent attribute
      highway_type:: string(get("highway"))
    ];

    create building from: shape_file_buildings;
    create intersection from: shape_file_nodes;
  }
}
```

2.2 Robust Attribute Mapping (Handling Missing/Malformed Shapefile Fields)

Real-world OSM shapefiles frequently have null, missing, or inconsistent attribute columns. Never assume a field exists — always guard it.

```
species road {
  int lanes <- 1;
  float capacity;
  float speed_limit <- 50 #km/#h;

  init {
    // get() returns nil if the attribute/column is absent -> guard every cast
    if (lanes = nil or lanes <= 0) {
      lanes <- 1;          // safe fallback
    }
    capacity <- lanes * 20.0;          // vehicles per lane, arbitrary
    calibration constant

    // string-based attribute you can't trust to always be numeric
  }
}
```

```

    string raw_speed <- string(self get_value_of "maxspeed");
    if (raw_speed != nil and raw_speed != "") {
        speed_limit <- float(raw_speed) #km/#h;
    }
}
}

```

Pattern: wrap every get("field") read in an if (... = nil) fallback branch. A model that crashes on the first malformed row of a 50,000-row OSM export is not production-grade.

2.3 Best Practices for Multi-Layer Import

```

global {
  // 1. Load ALL layers before creating any agents that reference each other
  file shape_file_roads <- file("../includes/roads.shp");
  file shape_file_buildings <- file("../includes/buildings.shp");
  file shape_file_nodes <- file("../includes/nodes.shp");

  geometry shape <- envelope(shape_file_roads);

  init {
    create road from: shape_file_roads;
    create building from: shape_file_buildings;
    create intersection from: shape_file_nodes;

    // 2. Snap topologically-close intersection nodes to road endpoints
    // (real OSM data is rarely perfectly noded)
    ask intersection {
      road nearest_road <- road closest_to self;
      if (nearest_road != nil and (self distance_to nearest_road) > 5#m) {
        write "WARNING: intersection " + self + " is " +
          (self distance_to nearest_road) + "m from nearest road - check topology.";
      }
    }
  }
}

```

```
}  
}  
}
```

Rules of thumb

- Load layers in dependency order: roads → nodes/intersections → buildings → any POI layer that snaps to roads.
- Always reproject all shapefiles to the same CRS before bringing them into GAMA (GAMA does not silently reconcile mismatched projections).
- Keep raw shapefiles read-only in ../includes/; never write back to them.

3. Graph Generation & Movement Engine

3.1 From Geometries to a Routable Graph

```
global {
  graph road_network;

  init {
    create road from: shape_file_roads;

    // Build a topological graph directly from the road agents' geometries.
    // Each road agent becomes an edge; endpoints become implicit nodes.
    road_network <- as_edge_graph(road);

    // Defensive check: verify the graph is connected before relying on it
    if road_network = nil {
      error "FATAL: road_network graph failed to build – check road geometries for
validity.";
    }
  }
}
```

For richer driving logic (lane count, priority, turn restrictions), build the graph explicitly for the `advanced_driving` skill:

```
global {
  graph road_network;
  init {
    road_network <- as_driving_graph(road, intersection);
    // as_driving_graph(edge_species, node_species) wires nodes AND edges together,
    // enabling right-of-way / intersection-aware routing.
  }
}
```

3.2 Vehicle Movement Skills

Option A — skill: moving (lightweight, generic agents)

```
species vehicle skills: [moving] {
  float speed <- (30 #km/#h);
  point target;

  reflex select_target when: target = nil {
    target <- any_location_in(one_of(road));
  }

  reflex drive when: target != nil {
    do goto target: target on: road_network move_weights: road_weights recompute_path: false;
    if (location = target) { target <- nil; } // arrived -> pick a new destination next
  }
  cycle
}
```

Option B — skill: `advanced_driving` (realistic car-following, lane-based)

```

species vehicle skills: [advanced_driving] {
  init {
    vehicle_length <- 3.0 #m;
    max_speed <- (30 + rnd(20)) #km/#h;
    max_acceleration <- 1.5;
    security_distance_coeff <- 1.5; // built-in car-following buffer
    right_side_driving <- true;
  }

  reflex select_next_path when: current_path = nil {
    intersection start_n <- one_of(intersection);
    intersection end_n <- one_of(intersection);
    current_path <- compute_path(graph: road_network, source: start_n.location, target:
end_n.location);
  }

  reflex move when: current_path != nil {
    do drive; // built-in primitive: handles acceleration, lane choice, and car-following
automatically
  }
}

```

3.3 Dynamic Pathfinding, Weighted Edges, and Re-routing

```

global {
  map<road, float> road_weights; // dynamic weight map used for congestion-aware routing

  reflex update_weights {
    // Recompute travel-time-like weights each N cycles based on live congestion
    road_weights <- road as_map (each :: each.shape.perimeter / max(0.1, each.avg_speed));
  }
}

species vehicle skills: [moving] {
  reflex reroute when: (every(20#cycle)) {
    // Periodically recompute the shortest path using UP-TO-DATE weights,
    // simulating drivers reacting to congestion (like live traffic-aware GPS)
    path new_path <- path_between(road_network, location, target);
    if (new_path != nil) {
      current_path <- new_path;
    }
  }

  reflex drive {
    do goto target: target on: road_network move_weights: road_weights recompute_path: false;
  }
}

```

Disconnected-graph safety pattern:

```

reflex select_target when: target = nil {
  point candidate <- any_location_in(one_of(road));
  path p <- path_between(road_network, location, candidate);
  if (p != nil and length(p.edges) > 0) {
    target <- candidate;
  }
}

```

```
}  
  // else: silently retry next cycle instead of crashing on an unreachable node  
}
```

4. Traffic Flow, Interactions & Micro-behaviors

4.1 road Species

```
species road {
  int lanes <- 1;
  float capacity;           // max vehicles the segment can hold
  float speed_limit <- 50 #km/#h;
  list<vehicle> current_vehicles <- [];
  float avg_speed <- speed_limit;
  float congestion_coeff <- 0.0; // 0 = free flow, 1 = gridlock

  reflex compute_congestion {
    current_vehicles <- vehicle inside self;
    int nb <- length(current_vehicles);
    congestion_coeff <- min(1.0, nb / max(1.0, capacity));
    avg_speed <- speed_limit * (1 - 0.8 * congestion_coeff); // simple linear speed-density
  }

  model
  {

    aspect base {
      draw shape color: (congestion_coeff > 0.7 ? #red : (congestion_coeff > 0.3 ? #orange :
#green)) width: lanes;
    }
  }
}
```

4.2 vehicle Species — Acceleration, Following Distance, Re-routing

```
species vehicle skills: [moving] {
  float speed_max <- (30 + rnd(30)) #km/#h;
  float speed <- speed_max;
  float acceleration <- 1.5;
  float following_distance <- 8.0 #m;
  point target;
  road current_road;
  rgb color <- rnd_color(255);

  reflex accelerate {
    speed <- min(speed_max, speed + acceleration);
  }

  reflex avoid_collision {
    list<vehicle> ahead <- (vehicle at_distance following_distance)
                          where (each != self and (self towards each) < 30);
    if not empty(ahead) {
      speed <- speed * 0.3; // brake sharply if something is close ahead
    }
  }

  reflex drive when: target != nil {
    do goto target: target on: road_network speed: speed;
    current_road <- road(agent_closest_to(self, road));
  }
}
```

```

    aspect base {
      draw triangle(6) color: color rotate: heading;
    }
  }
}

```

4.3 intersection / traffic_light Species

```

species intersection {
  aspect base {
    draw circle(2) color: #gray;
  }
}

species traffic_light parent: intersection {
  int cycle_duration <- 30 #cycle;           // total cycle length
  int green_duration  <- 15 #cycle;          // portion of cycle that is green
  string state <- "green" among: ["green", "red"];
  list<road> controlled_roads <- [];

  reflex switch_state {
    int t <- cycle mod cycle_duration;
    state <- (t < green_duration) ? "green" : "red";
  }

  aspect base {
    draw circle(3) color: (state = "green" ? #green : #red);
  }
}

```

4.4 Vehicles Detecting Red Lights, Queuing, and Avoiding Overlap

```

species vehicle skills: [moving] {
  traffic_light next_light;
  float stop_distance <- 10 #m;

  reflex detect_light when: target != nil {
    next_light <- traffic_light closest_to self;
    if (next_light != nil and (self distance_to next_light) < stop_distance) {
      if (next_light.state = "red") {
        speed <- 0.0; // stop at red
      } else {
        speed <- speed_max; // proceed at green
      }
    }
  }
}

// Avoid overlapping / rear-ending the vehicle directly ahead on the same road segment
reflex maintain_gap when: current_road != nil {
  list<vehicle> same_road_ahead <- (current_road.current_vehicles)
    where (each != self and (each distance_to self) < following_distance
      and (self towards each) < 45);
  if not empty(same_road_ahead) {
    vehicle nearest <- same_road_ahead with_min_of (each distance_to self);
    speed <- min(speed, nearest.speed); // match the speed of the car ahead
  }
}

```

```
}  
}  
}
```

5. Visuals & Real-time Analytics

5.1 Multi-Layer 2D + 3D Display

```
experiment MainExperiment type: gui {
  output {
    display Map_2D type: opengl {
      species road aspect: base;
      species building aspect: base;
      species intersection aspect: base;
      species traffic_light aspect: base;
      species vehicle aspect: base;
    }

    display Map_3D type: opengl {
      species road aspect: base;

      // extrude buildings using an attribute (e.g., number of floors * floor height)
      species building aspect: base3D;

      species vehicle aspect: base3D;

      light #ambient intensity: 100;
    }
  }
}

species building {
  float height <- 10.0 + rnd(20.0);

  aspect base {
    draw shape color: #lightgray;
  }
  aspect base3D {
    draw shape depth: height color: #lightgray border: #darkgray;
  }
}

species vehicle skills: [moving] {
  aspect base3D {
    draw pyramid(3) color: color rotate: heading;
  }
}
```

5.2 Monitors and Live Charts

```
global {
  int total_trips_completed <- 0;
  float total_trip_duration <- 0.0;
  float avg_trip_duration <- 0.0;
  float global_congestion_index <- 0.0;

  reflex compute_kpis {
    global_congestion_index <- mean(road collect each.congestion_coeff);
  }
}
```

```

        avg_trip_duration <- (total_trips_completed > 0) ? (total_trip_duration /
total_trips_completed) : 0.0;
    }
}

experiment MainExperiment type: gui {
  output {
    // ---- MONITORS (single live-value gauges) ----
    monitor "Vehicles on road"      value: length(vehicle);
    monitor "Global congestion index" value: global_congestion_index;
    monitor "Avg trip duration (s)"  value: avg_trip_duration;

    // ---- LIVE CHARTS ----
    display Analytics refresh: every(5#cycle) {
      chart "Vehicle Count Over Time" type: series {
        data "nb_vehicles" value: length(vehicle) color: #blue;
      }
    }

    display Congestion_Chart {
      chart "Congestion Index Over Time" type: series style: line {
        data "congestion" value: global_congestion_index color: #red;
      }
    }

    display Trip_Duration_Chart {
      chart "Average Trip Duration" type: series style: line {
        data "avg_duration" value: avg_trip_duration color: #darkgreen;
      }
    }
  }
}

```

6. Complete, Ready-to-Run Single-File Template

Copy the block below into `RoadTraffic.gaml` inside a GAMA project with a `models/` folder and an `includes/` subfolder containing `roads.shp`, `buildings.shp`, and `nodes.shp` (all in the same CRS, ideally EPSG:4326 or a local projected CRS).

• GLOBAL BLOCK

```
/**
 * Name: RoadTrafficModel
 * Description: End-to-end GIS-based road traffic simulation.
 *              Loads real-world shapefiles, builds a routable graph,
 *              simulates vehicles with car-following + traffic lights,
 *              and reports live KPIs.
 * Tags: traffic, GIS, graph, driving
 */

model RoadTrafficModel

global {

  // -----
  // 1. GIS FILES
  // -----
  file shape_file_roads      <- file("../includes/roads.shp");
  file shape_file_buildings  <- file("../includes/buildings.shp");
  file shape_file_nodes      <- file("../includes/nodes.shp");

  geometry shape <- envelope(shape_file_roads);

  // -----
  // 2. GRAPH & GLOBAL STATE
  // -----
  graph road_network;
  map<road, float> road_weights;

  int nb_vehicles_init <- 50 min: 1 max: 1000;

  int total_trips_completed <- 0;
  float total_trip_duration <- 0.0;
  float avg_trip_duration <- 0.0;
```

```

float global_congestion_index <- 0.0;

init {
  // ---- Defensive load checks ----
  if (shape_file_roads = nil) { error "FATAL: could not load roads.shp";
}
  if (shape_file_nodes = nil) { error "FATAL: could not load nodes.shp";
}

  create road from: shape_file_roads with: [
    lanes:: int(get("lanes")),
    speed_limit_raw:: string(get("maxspeed"))
  ];

  create building from: shape_file_buildings;
  create intersection from: shape_file_nodes;

  // Promote a fraction of intersections to traffic_light
  ask (intersection count(true) > 0 ? intersection : []) {
    if (flip(0.3)) {
      create traffic_light with: [location:: self.location];
      do die; // replace plain intersection with a controlled one
    }
  }

  // ---- Build the routing graph, with a fallback check ----
  road_network <- as_edge_graph(road);
  if (road_network = nil) {
    error "FATAL: road_network failed to build - check for
invalid/duplicate road geometries.";
  }

  // ---- Spawn vehicles ----
  create vehicle number: nb_vehicles_init {
    location <- any_location_in(one_of(road));
  }
}

reflex update_weights when: every(10#cycle) {
  road_weights <- road as_map (each :: each.shape.perimeter / max(0.1,
each.avg_speed));
}

reflex compute_kpis {
  global_congestion_index <- (length(road) > 0) ? mean(road collect
each.congestion_coeff) : 0.0;
  avg_trip_duration <- (total_trips_completed > 0) ?
(total_trip_duration / total_trips_completed) : 0.0;
}
}

• SPECIES – road, building, intersection, traffic_light
// =====
// SPECIES
// =====

```

```

species road {
  int lanes <- 1;
  string speed_limit_raw;
  float speed_limit <- 50 #km/#h;
  float capacity;
  list<vehicle> current_vehicles <- [];
  float avg_speed;
  float congestion_coeff <- 0.0;

  init {
    if (lanes = nil or lanes <= 0) { lanes <- 1; }
    if (speed_limit_raw != nil and speed_limit_raw != "") {
      speed_limit <- float(speed_limit_raw) #km/#h;
    }
    capacity <- lanes * 15.0;
    avg_speed <- speed_limit;
  }

  reflex compute_state {
    current_vehicles <- vehicle inside self;
    int nb <- length(current_vehicles);
    congestion_coeff <- min(1.0, nb / max(1.0, capacity));
    avg_speed <- speed_limit * (1 - 0.8 * congestion_coeff);
  }

  aspect base {
    draw shape color: (congestion_coeff > 0.7 ? #red : (congestion_coeff >
0.3 ? #orange : #green)) width: lanes + 1;
  }
}

species building {
  float height <- 10.0 + rnd(20.0);
  aspect base { draw shape color: #lightgray border: #gray; }
  aspect base3D { draw shape depth: height color: #lightgray border:
#darkgray; }
}

species intersection {
  aspect base { draw circle(2) color: #darkgray; }
}

species traffic_light parent: intersection {
  int cycle_duration <- 30;
  int green_duration <- 15;
  string state <- "green" among: ["green", "red"];

  reflex switch_state {
    int t <- cycle mod cycle_duration;
    state <- (t < green_duration) ? "green" : "red";
  }

  aspect base {
    draw circle(3) color: (state = "green" ? #green : #red);
  }
}

```

- SPECIES – vehicle

```
species vehicle skills: [moving] {
  float speed_max <- (30 + rnd(30)) #km/h;
  float speed <- speed_max;
  float acceleration <- 1.5;
  float following_distance <- 8.0 #m;
  point target;
  road current_road;
  rgb color <- rnd_color(255);
  int start_time <- 0;

  init {
    start_time <- cycle;
  }

  reflex select_target when: target = nil {
    point candidate <- any_location_in(one_of(road));
    path p <- path_between(road_network, location, candidate);
    if (p != nil and length(p.edges) > 0) {
      target <- candidate;
    }
  }

  reflex detect_light when: target != nil {
    traffic_light nearest_light <- (traffic_light closest_to self);
    if (nearest_light != nil and (self distance_to nearest_light) < 10#m
        and nearest_light.state = "red") {
      speed <- 0.0;
    } else {
      speed <- speed_max;
    }
  }

  reflex maintain_gap when: current_road != nil {
    list<vehicle> ahead <- (current_road.current_vehicles)
      where (each != self and (each distance_to self) <
following_distance
      and (self towards each) < 45);
    if not empty(ahead) {
      vehicle nearest <- ahead with_min_of (each distance_to self);
      speed <- min(speed, nearest.speed);
    }
  }

  reflex drive when: target != nil {
    do goto target: target on: road_network move_weights: road_weights
recompute_path: false;
    current_road <- road(road closest_to self);

    if (location = target) {
      total_trips_completed <- total_trips_completed + 1;
      total_trip_duration <- total_trip_duration + (cycle - start_time);
      target <- nil;
      start_time <- cycle;
    }
  }
}
```

```

    aspect base {
      draw triangle(6) color: color rotate: heading;
    }
    aspect base3D {
      draw pyramid(3) color: color rotate: heading;
    }
  }
}

• EXPERIMENT

// =====
// EXPERIMENT
// =====

experiment MainExperiment type: gui {
  parameter "Initial number of vehicles" var: nb_vehicles_init category:
  "Traffic";

  output {
    display Map_2D type: opengl background: #white {
      species road aspect: base;
      species building aspect: base;
      species intersection aspect: base;
      species traffic_light aspect: base;
      species vehicle aspect: base;
    }

    display Map_3D type: opengl background: #black {
      species road aspect: base;
      species building aspect: base3D;
      species vehicle aspect: base3D;
    }

    monitor "Vehicles on road" value: length(vehicle);
    monitor "Global congestion index" value: global_congestion_index;
    monitor "Avg trip duration (cycles)" value: avg_trip_duration;

    display Analytics refresh: every(5#cycle) {
      chart "Vehicle Count Over Time" type: series {
        data "nb_vehicles" value: length(vehicle) color: #blue;
      }
    }

    display Congestion_Chart {
      chart "Congestion Index Over Time" type: series style: line {
        data "congestion" value: global_congestion_index color: #red;
      }
    }
  }
}

```

Final Notes for Production Hardening

- Always validate CRS alignment across roads.shp, buildings.shp, nodes.shp before import — mismatched projections silently produce a broken envelope().
- Guard every get("field") with a nil/empty check — real OSM exports are inconsistent.

- Check `as_edge_graph(...)` / `as_driving_graph(...)` for `nil` immediately after creation, and `error(...)` fast rather than letting `goto` silently fail on a broken topology.
- Prefer `advanced_driving` skill over plain moving once you need lane-level realism, priority rules at intersections, or overtaking behavior — it costs more performance but eliminates a lot of hand-rolled car-following code shown in Section 4.
- Use batch experiments (Section 1.3) once the GUI model is stable, to calibrate `speed_limit`, `capacity`, and `security_distance_coeff` against real traffic counts.